



Estrutura dos Sistemas Operacionais

Prof. Pedro Botelho

Universidade Federal do Ceará

17 de março de 2026



Interrupções e Privilégios

Chamadas de Sistema

Conclusão

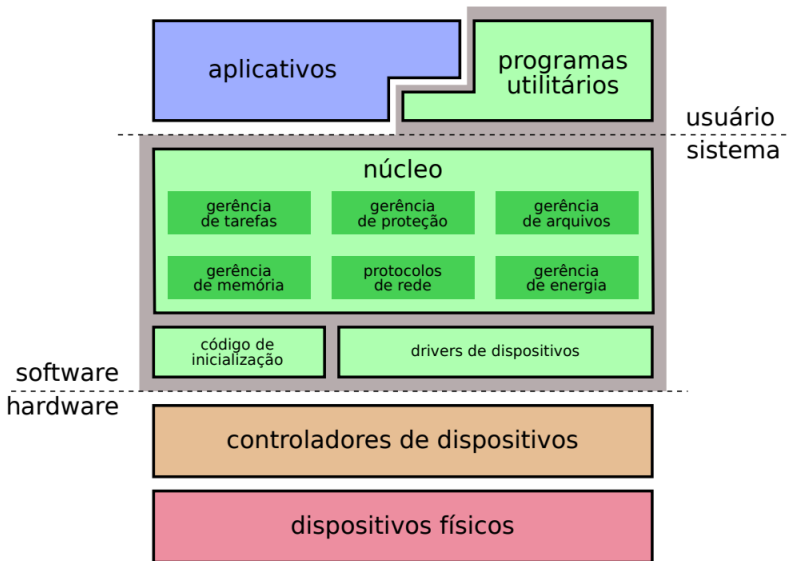


Interrupções e Privilégios

Chamadas de Sistema

Conclusão

Relembre: Estrutura do Sistema





Política

Aspecto abstrato de alto nível: decidir a quantidade de memória para cada aplicação, o próximo pacote de rede a enviar, etc...

Mecanismo

Procedimento de baixo nível usado para implementar políticas: como iniciar um processo, enviar um pacote de rede, etc...

Filosofia da estrutura: separar políticas de mecanismos

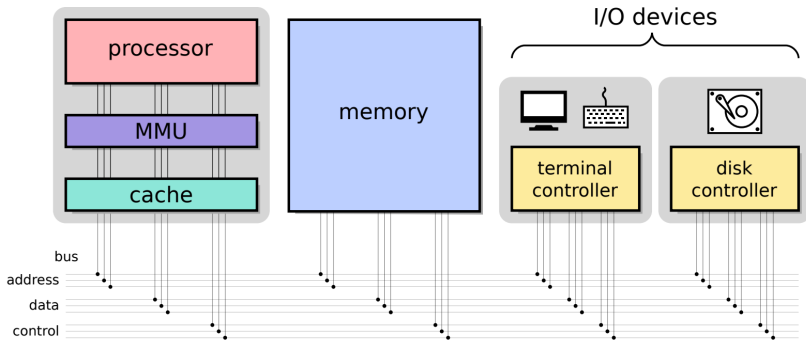
- ▶ As políticas devem ser **independentes** dos mecanismos
- ▶ Os mecanismos devem ser **genéricos** para várias políticas

Relembre: Estrutura de *Hardware*



Processador interage com memória e dispositivos de E/S por meio de barramentos.

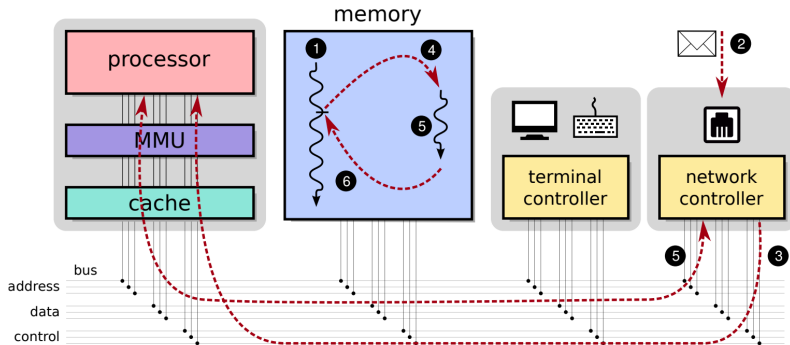
- ▶ Executa um programa (conjunto de instruções) na memória
- ▶ Acessa dispositivos de E/S por meio de controladores





- ▶ Eventos internos e externos devem ser tratados assim que ocorrerem
 - ▶ Mecanismos de *hardware* desviam a execução do processador para o tratador destes eventos (código especializado)
- ▶ **Interrupção**: Desvia a execução por evento externo (assíncrono), gerado por um periférico e.g. clique do mouse
- ▶ **Exceção**: Desvia a execução por evento interno (síncrono) e.g. divisão por zero
 - ▶ Em resposta a uma instrução → Impede a execução normal
 - ▶ Geralmente um erro, mas nem sempre e.g. *breakpoint*
- ▶ **Trap**: Exceção que desvia a execução a pedido do *software*
 - ▶ Intencional → Instrução específica ativa o evento interno

Exemplo: Interrupção de Rede



Roteiro de uma Interrupção



1. O processador está em um fluxo de execução
2. Um pacote vindo da rede é recebido pela placa Ethernet
3. O controlador Ethernet envia uma IRQ ao processador
4. O processamento é desviado para a rotina de tratamento da interrupção
5. A rotina de tratamento transfere os dados do pacote do controlador para a memória
6. A rotina de tratamento finaliza e o processador retorna à execução do programa

Interrupções Típicas do x86



IRQ	Descrição
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
...	...
19-31	Intel reserved
32-255	maskable interrupts (devices & exceptions)



Conhecido como espaço de kernel ou *kernel space*

- ▶ Acesso **amplo** aos recursos:
 - ▶ todas as instruções do processador
 - ▶ todos os registradores
 - ▶ todas as portas de entrada/saída
 - ▶ todas as áreas da memória
- ▶ Nível usado pelo núcleo e pelos drivers.
- ▶ Usado também pelo supervisor e monitor



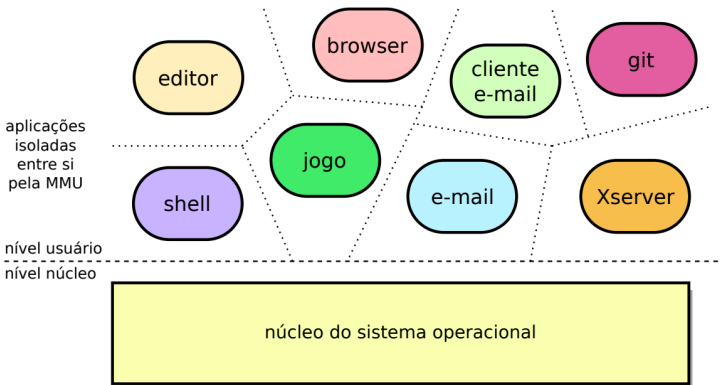
Conhecido como espaço de usuário ou *userspace*

- ▶ Acesso **restrito** aos recursos:
 - ▶ subconjunto das instruções do processador
 - ▶ subconjunto de registradores
 - ▶ subconjunto de portas de entrada/saída
 - ▶ subconjunto de áreas da memória
- ▶ Tentativas de acesso inválidas geram exceções
- ▶ Nível usado pelos utilitários e aplicações
- ▶ Como executar ações privilegiadas?

Isolamento



- ▶ Núcleo (*kernel*) isolado das aplicações. Por que?
- ▶ Aplicações isoladas entre si. Por que?



Conteúdo



Interrupções e Privilégios

Chamadas de Sistema

Conclusão

Chamadas de Sistema (ou *syscall*)



São funções que permitem o acesso aos serviços do núcleo:

- ▶ Abrir/ler/escrever/fechar arquivos
- ▶ Enviar/receber dados através da rede
- ▶ Ler teclado
- ▶ Escrever dados na tela

Interface fornecida às aplicações → **Abstração**

Função **Trap** → Troca de modo

Problema

Como uma aplicação pode invocar uma função do núcleo?

Invocação de uma *syscall* (em C)



- ▶ Bibliotecas de rotinas → `unistd.h` fornece acesso à API do SO
 - ▶ POSIX → Define interface mínima e portátil
 - ▶ Permite que um programa desenvolvido para um SO seja facilmente portado para outro
- ▶ Exemplo: Transferência de blocos de dados

```
1  #include <unistd.h>
2
3  int main (int argc, char *argv[])
4  {
5      write(1, "Hello World!\n", 13); // write string to stdout
6      exit(0); // exit with no error
7  }
```

Retorno de uma *syscall*



Em caso de erro, *syscall* sempre retorna -1 e código de erro atribuído a *errno* (variável global, impresso via *perror*).

```
1  #include <unistd.h>
2
3  int main (int argc, char *argv[])
4  {
5      int c = write(1, "Hello World!\n", 13);
6      if(c == -1) {
7          perror("ERROR\n");
8          exit(1);
9      }
10     // ...
11     exit(0);
12 }
```

Atenção: Programas devem sempre verificar o retorno de uma chamada de sistema!

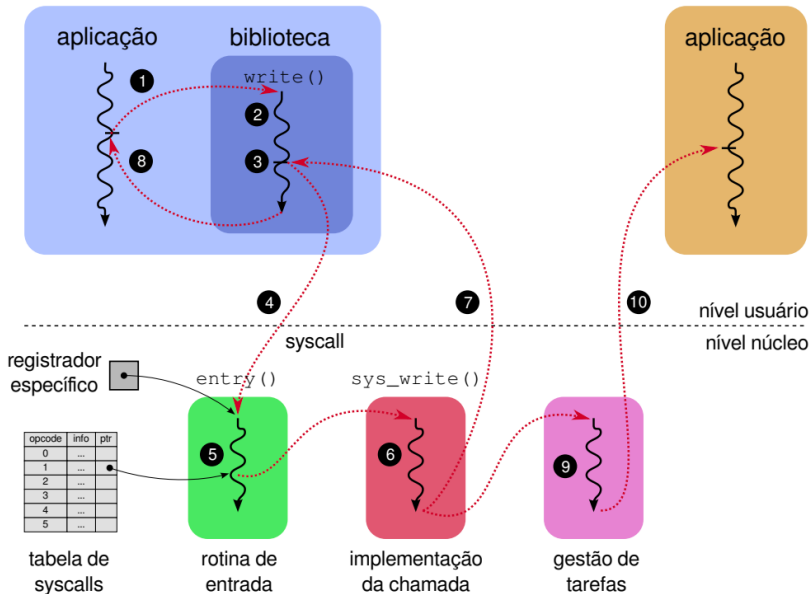
Invocação de uma syscall (em Assembly)



Funções em C $\rightarrow$ *Wrappers* para instruções Assembly

```
1  section .data
2  msg db    Hello    World!    , 0xA ; output string (0xA = "\n")
3  len equ 13 ; string size
4
5  section .text
6
7  global _start
8
9  _start:
10 mov rax, 1 ; syscall opcode (1: write)
11 mov rdi, 1 ; file descriptor (1: stdout)
12 mov rsi, msg ; data to write
13 mov rdx, len ; number of bytes to write
14 syscall ; make syscall (Trap)
15
16 mov rax, 60 ; syscall opcode (60: _exit)
17 mov rdi, 0 ; exit status (0: no error)
18 syscall ; make syscall (Trap)
```

Etapas de uma Chamada de Sistema

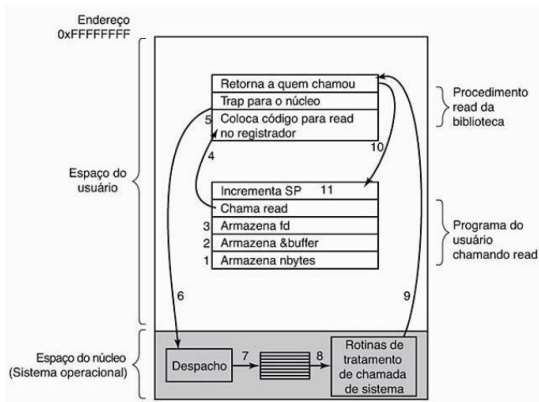


Etapas de uma Chamada de Sistema



1. Aplicação chama `write` da biblioteca.
2. A função `write` preenche os registradores da CPU.
3. A função `write` invoca uma chamada de sistema.
4. A CPU vai para o *kernel* e ativa a rotina de entrada (`entry`).
5. A rotina de entrada consulta a tabela de *syscalls* e ativa a função `sys_write`.
6. A função `sys_write` efetua a operação solicitada.
7. A CPU retorna à função `write`, em modo usuário.
8. A função `write` retorna ao código principal da aplicação.

Outra visualização...



- ▶ Armazena parâmetros na pilha
- ▶ Chamada a rotina da biblioteca
- ▶ Coloca o número da chamada em um local esperado pelo SO
- ▶ Executa instrução TRAP
- ▶ Tabela → rotina de tratamento
- ▶ Retorno à rotina da biblioteca
- ▶ Programa limpa a pilha

Conjunto de Chamadas de Sistema



Processos: criar, carregar código, terminar, esperar

Memória: alocar/liberar/modificar áreas de memória

Arquivos: criar, remover, abrir, fechar, ler, escrever

Comunicação: criar/destruir canais, receber/enviar dados

Dispositivos: ler/mudar configurações, ler/escrever dados

Sistema: ler/mudar data e hora, desligar o sistema

Cada SO define seu próprio conjunto de *syscalls*:

1. Sistemas Windows: **Win32**
2. Sistemas UNIX: **POSIX**

Exemplo: *Syscall* no Gerenciamento de Processos



- ▶ Chamada *fork*: criação de processo filho
 - ▶ Variáveis iguais após criação
 - ▶ Mudanças independentes
- ▶ Exemplo: *shell*
 - ▶ Lê comando do terminal
 - ▶ Cria processo filho
 - ▶ Espera que o filho execute o comando (*waitpid*)
 - ▶ Após terminado o processo filho, executa próximo comando

Exemplo: Syscall no Gerenciamento de Processos



```
#define TRUE 1

while (TRUE) {
    type_prompt( );
    read_command(command, parameters);

    if (fork() != 0) {
        /* Codigo do processo pai. */
        waitpid(-1, &status, 0);
    } else {
        /* Codigo do processo filho. */
        execve(command, parameters, 0);
    }
}
```

/* repita para sempre */
/* mostra prompt na tela */
/* le entrada do terminal */

/* cria processo filho */

/* aguarda o processo filho acabar */

/* executa o comando */

Conteúdo



Interrupções e Privilégios

Chamadas de Sistema

Conclusão



- ▶ Políticas (alto nível) X Mecanismos (baixo nível)
- ▶ Interrupção (externo) X Exceções (interno)
- ▶ Espaço de Usuário X Núcleo (*kernel*)
- ▶ Chamada de Sistema → Serviços do núcleo
- ▶ API do Sistema Operacional → Conjunto de *syscalls*

Dúvidas?



Obrigado pela atenção!